

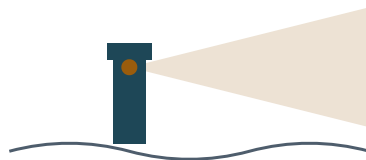
Lighting the Storm

An apprentice keeper repairs a failing tower, lets two storm-battered sailors in for shelter, and crosses a worsening sea for the one part that can't be fixed on site, before a passenger ship is lost on the rocks.

By **Paul** · Updated **1 October 2026** · post-review

First-person · single-player · PC (Steam)

Bracken Isle Light · Fl(2) W 15s



PART 1

Executive Summary

A 30-minute first-person atmospheric adventure about keeping a light alive through one storm, built by a solo developer with AI assistance. A 10-minute prototype comes first, over six weeks.

UPDATED AFTER DESIGN REVIEW

This version reflects the six-reviewer design review: the rowboat crash is replaced by the sailors seeking shelter; the deadline starts when the return crossing begins and the ship is introduced at boarding; storm perception and hazard density are separated; a fail-state contract defines what the player perceives and why they failed; the discoveries move before boarding; Ted is staged at the lamp; the Gate 1 slice has two repairs and restarts on a fail; and AI orchestrators oversee the delegated work.

Pitch and player promise

You are the new apprentice to Ted, keeper of a remote, storm-battered lighthouse on Bracken Isle. You work through the building's failing systems under his direction, but the beacon itself won't stop flickering. Two sailors, Jeff and Daniel, overwhelmed by the storm, moor their rowboat on the island and look for shelter. Ted sees them from the window and tells you to open the island gate so they can come inside. With Jeff beside you, who has realised the storm is turning nasty and wants to stop the passenger ship being wrecked, you cross the storm in an engine boat to fetch a replacement burner from an abandoned outpost, then race back to fit it before the ship reaches the rocks.

The central player skill is steering a powered boat past rocks and other hazards while the storm worsens. The experience should deliver sustained dread, hands-on mechanical work, and a consequential ending. There is no combat, no monster reveal and no jump scares. Folk-horror details invite interpretation without ever confirming a supernatural cause.

Design pillars

Every feature and story beat is judged against four pillars. Each has a test question used in reviews and cuts.

Pillar 1

The light must return

Everything leads back to the beacon. It is what you repair, it guides you home, and it decides whether the ship lives.

Test: does this connect to getting the light back, or to what happens if it fails?

Pillar 2

The storm sets the clock

Pressure comes from the world: the approaching ship's lights, worsening weather, falling visibility and rocks in the dark. Never a timer, meter or enemy. Elapsed time drives what the player perceives; phase and route position drive what they collide with.

Test: can the player feel time running out without any UI?

Pillar 3

Your hands, your eyes

Everything is first-person and physical. You fix things by hand, see the sailors arrive from the window, and see or hear the ship's fate from where you stand. Guidance comes from Ted and Jeff, not markers.

Test: is this happening to the player in the world, or being shown to them?

Pillar 4

Dread you can read

Sustained unease and ambiguity, never jump scares or a confirmed monster. The player always knows what to do and why they failed: one steering control, clear hazards, fair and recoverable mistakes, and a stated reason on failure.

Test: does this add unease without adding confusion?

Scope rule: small world, read closely. One lighthouse, one stretch of sea, one abandoned outpost, and no new major locations. A feature that fails a pillar's test, or expands the world without deepening the main loop, is deferred.

Cast

- **The Apprentice (player).** Characterised through action and a handwritten objective journal rather than dialogue trees. The apprentice never speaks, so others carry the story's moral beats.
- **Ted, the keeper.** Late 50s, thirty years in the tower. Gruff and blunt, with a guarded sense of care. He directs the repairs out loud, sees the sailors from the window, names the ship at the boarding

threshold, and waits at the lamp for the final repair and the outcome (voice and position only). His presence is to be locked by the end of week 3.

- **Jeff, the older seaman.** Hard, coarse and unhappy about helping, but his instinct is to keep people alive. He has realised the storm is getting nasty and wants to stop the ship being wrecked. He is the player’s companion on both crossings and calls out navigation warnings; after the second crossing he tells the player to go on without him.
- **Daniel, the younger seaman.** Shaken by the storm and just glad to be alive. He goes to the basement and stays there; his reaction to the outcome is not recorded.

Scope

	First playable prototype	Full game
Length	About 10 minutes	About 30 minutes
Content	Two repairs, the moored rowboat seen from the window, island gate, boarding with Jeff, outbound crossing, burner pickup, return crossing, burner fitting, win and fail outcomes	Full repair set, sailors and gate, two crossings, lighthouse and island discoveries, a lighter outpost set, win and fail outcomes
Art and audio	Greybox art, placeholder audio	Final art, audio and dialogue
Schedule	6 weeks, solo, AI-assisted	A further 5–6 months (estimate)
Purpose	Prove that hazards are readable and steering is enjoyable	Commercial release on Steam, with controller support
After a fail	Restart the slice	Final retry and save model settled before Gate 2

STRETCH GOAL

Wave hazards that push or roll the boat are a stretch goal for both the prototype and the full game. The crossings are designed to work fully without them. Waves are added only after steering passes its week-2 go/no-go test and only if the schedule has room, and they are the first thing cut if it slips.

PROVISIONAL CUT LIST

The review found no schedule or estimate beyond the prototype. To be completed after Gate 1 playtests with measured data. First to cut, in order: wave hazards; return-crossing route branching; repairs beyond the first two; outpost optional discoveries beyond the light set.

Outcome checks

Prototype

- A new player understands what to repair, where to head, and which hazard to avoid, with no timer UI.
- The boat has one steering control, one fixed speed, clear feedback for near misses and collisions, and a recoverable error state.
- The ship is introduced at the boarding threshold, and its lights and position communicate the stakes.
- Both the win and fail outcomes can be reached, a failed player can say why they failed, and a fail restarts the slice.

Full game

- A first-time playthrough lasts about 30 minutes.
- The final game preserves unexplained environmental wrongness.
- The final retry and save model is decided and tested for unwinnable saves.

Game Mechanics

What the player does, in the order they do it, and the systems that shape each action.

Core loop

The loop mirrors one worsening night on duty and plays once through the main story.

- 01 **Repair the lighthouse.** Ted barks which system is broken and roughly where its part is. The player explores, diagnoses and fixes it with a fixed toolkit. The beacon alone keeps flickering. NO DEADLINE

- 02 **See the sailors, open the gate.** Ted calls the player to the window, where Jeff and Daniel's rowboat has moored on the island, the sailors overwhelmed by the storm. He tells the player to open the island gate; Jeff and Daniel make their own way inside, and Daniel goes down to the basement. Ted explains the burner must be fetched from the abandoned outpost. NO DEADLINE

- 03 **Board the engine boat with Jeff.** At the boarding threshold Ted names the passenger ship's lights as the reason for the errand. The outbound crossing is free steering practice, with no fail state. NO DEADLINE YET

- 04 **Cross to the outpost.** Steer past rocks, buoys and debris in the storm while Jeff shouts directions.

- 05 **Pick up the burner.** Find it at the outpost and get it aboard.

- 06 **Cross back.** The storm peaks: denser hazards, lower visibility, fragile cargo aboard. The hidden deadline starts as the boat sets off. DEADLINE STARTS

- 07 **Fit the burner.** Climb to the lamp room and install it as the ship's lights close on the rocks. Ted waits at the lamp; Jeff has told the player to go on without him.

- 08 **Win or fail.** Beacon lit in time: the ship sails calmly into port. Deadline missed: the ship crashes into the island. OUTCOME

Lighthouse repairs

The lighthouse runs on interdependent systems: the lamp and optic assembly, the clockwork rotation mechanism, the fuel and fog-signal lines, and stormproofing (shutters and seals). Each broken system is fixed through a short, first-person puzzle, such as rewiring a junction box, swapping a gear or patching a leak, using a small fixed toolkit rather than a crafting menu. The prototype includes two repairs; further repairs belong to Gate 2 and the full game. The core folk-

horror discoveries sit in the lighthouse and on the island, before boarding and outside the deadline, tied to why the beacon failed; the outpost keeps a lighter set, and no significant outpost detail is added until the prototype is considered complete.

- **Ted's guidance** is the first assist mechanism. He names the part and roughly where to find it; how specific he is scales with the difficulty preset.
- **The journal** is a handwritten-style checklist of what is broken, updated as systems are diagnosed. It reflects progress but never controls it.
- **Gating.** Some repairs open new areas, for example repairing the winch opens the hatch to the boat dock. This gives one building light Metroidvania-style gating without a large map.
- **The beacon as inciting incident.** Once every other system works, the beacon still flickers. That unresolved fault leads Ted to see the sailors on the island, so a mechanical dead end becomes the story's turn instead of a quest marker.

The sailors and the gate

The arrival is scripted and seen entirely in first person through the lighthouse window: Jeff and Daniel's rowboat is moored on the island, with no crash. It triggers when the window is in view, with a short fallback so it cannot be missed. Opening the island gate is a single interaction; Jeff and Daniel walk in on their own, with no escort, and Daniel goes to the basement and stays there. Opening the gate advances the story and opens the route to the engine boat, where Jeff waits.

Sea crossing and boat navigation

The player pilots a small engine boat at **one fixed speed with one control: steering**. There is no throttle and no oars. The challenge is steering past placed hazards (rocks, reefs, buoys and floating debris) along a route that narrows and darkens as the storm worsens. The sea surface moves for atmosphere but does not push the boat.

- **Jeff's calls** are the second assist mechanism. His warnings read from the same hazard data as the world, so they never disagree with the real threat. How early and how often he calls scales with the difficulty preset.
- **Navigation aids:** a compass, the lighthouse beam as a guiding light when heading home, and silhouettes of buoys and rocks in low visibility.
- **Risk states:** a collision takes on water, slows the boat, or causes a brief capsize and recovery at a nearby safe point, all costing time. On the return leg, rough handling can knock the burner loose, costing time to recover it. The burner is never permanently lost (proposed). Docking at each jetty is automatic because the boat cannot stop. Penalty rules are deliberately loose for the prototype and must be reviewed for the full game.
- **Stretch goal:** authored waves that push and roll the boat, added as an optional layer only if time allows.

Tension and the deadline

The central pressure is a countdown the player never sees as a number. It starts when the player has the burner aboard and sets off on the return crossing. Ted introduces the ship at the boarding threshold, but everything before the return crossing, including the outbound crossing and the

outpost, plays at its own pace with no fail state, so the player can get a feel for the boat. From then on, the approaching ship's lights on the horizon and the rising storm (wind, rain, lightning) communicate how much time is left. It is the only timer in the game: the same deadline runs through the return crossing and the final repair.

Most failures are soft. A collision costs time rather than ending the run, which keeps the player in the fiction. The exception is the deadline itself, which is a true fail state, so the ending means something.

Storm escalation

Two channels keep a struggling player from being punished twice. What the player *perceives* (sound, lightning, wind and the ship's lights) follows elapsed time. What the player *collides with* (hazard density, route width, hazard visibility) follows phase and route position only. Elapsed time never raises the hazards, so a slow player faces the same crossing as a fast one, with a darker and louder sky.

When the deadline expires

The sequence is always the foghorn, the ship crashing into the island, silence, then a “Failed” title screen with a short line saying why. The deadline runs only from the return crossing, so there are three fail states. What the player perceives depends on where they are, so no cutaway camera is needed.

Where the deadline expires	What the player perceives
On the return crossing	Audio only: ship horn, crash sounds, silence, then the Failed screen
In the lamp room	The player sees the ship hit the island from a clear lamp-room sightline, and the audio still sounds
Elsewhere in the lighthouse (stairs)	Audio only, as on the crossing, unless a window sightline exists

Ted reacts to either outcome from the lamp room by voice only. In the prototype a fail restarts the slice; the final retry and save model is settled before Gate 2, and must restore time and weather together so a reload can never be unwinnable.

Escalation curve

Act 1 · Lighthouse Calm, tutorialised, interrupted by the sailors' arrival	Act 2 · Outbound Moderate hazards, no deadline yet: free practice	Act 3 · Return Severe: the hardest skill test in the game	Climax · Burner A multi-step repair under the same deadline
--	---	---	---

Difficulty and accessibility

Difficulty is mainly a pacing and tension curve, not a skill gate. A single preset controls how specific Ted's directions are, how much Jeff helps on the water, and how intense the hazards are, without changing the story or outcomes. Navigation calls are subtitled with speaker names and an optional visual direction cue, and the audio-only fail cue has a subtitle or visual alternative. Steering and interaction are remappable, with adjustable sensitivity and a reduced-hazard preset.

Presentation

All viewpoints are first-person, with no cutaway or exterior cameras. Warm oil-lamp light indoors plays against blue-grey storm light outside, and the beacon's sweeping beam is the constant visual anchor. Sound is diegetic first: wind, rain, groaning timber, machinery, buoy bells and a distant foghorn, with no shock stingers. Strange symbols, unexplained shrines and architecture that doesn't quite fit suggest a history without ever explaining it.

AI Architecture

The game is built by one developer working with ten AI worker roles, each owning one department of the build, overseen by two orchestrator agents. Below, each worker is described through what the player experiences as a result of its work.

SCOPE OF AI USE

AI is used only during development. The shipped game makes no AI calls at runtime: Ted’s and Jeff’s lines, the storm and the ship are authored and scripted, so the game has no API dependency or per-player running cost.

Principles

- **One agent, one department.** Each agent works from a bounded task brief naming its goal, inputs, owned files, interfaces and acceptance checks.
- **The developer owns judgement.** Steering feel, tone, art fit, dialogue performance and story interpretation are human decisions. AI output is a first pass that the developer reviews and merges.
- **The pillars bind every agent.** Work that fails a pillar’s test is returned, whatever its technical quality.
- **No silent canon.** When a story or design choice is unresolved, the agent stops and asks instead of deciding. Decisions already made are recorded in a decisions log that every agent treats as canon.
- **Orchestrators delegate and oversee.** The developer gives goals to orchestrators, who split them into briefs, assign workers, order dependencies and check results. They escalate canon questions and never merge.

Orchestration and oversight

Orchestrator	Responsibility	Escalates to the developer when
Production Orchestrator	Turns a goal or gate into task briefs, assigns workers, orders dependencies, tracks the schedule, holds event IDs and contracts, assembles the integrated build	An exit test is missed, a contract must change, two tasks conflict, or week 5 slips
Canon and Review Orchestrator	Checks every handoff against the pillars, the scope rule and the decisions log; blocks work that decides unresolved story; keeps the decisions log current	A canon question appears (such as Ted, the retry model or penalty rules) or a pillar test fails

Each orchestrator reports what was delegated, what came back, what passed review, what failed and why, and which developer decisions are blocking. Only the developer merges.

The agents and their effect on play

Boat / Physics agent Serves **pillar 4** · the core skill

WHAT THE PLAYER EXPERIENCES

The feel of every crossing: how quickly the boat answers the stick, how clearly a near miss or collision registers, and how quickly the player recovers and gets back on course.

WHAT IT BUILDS

- Boat controller at one fixed speed
- Hazard collisions and recovery
- Tuning values held as data
- Stretch: wave layer behind an on/off setting

Review gate: the developer plays the week-2 steering spike and makes the go/no-go call on feel.

Level Layout agent Serves **pillar 3** and the scope rule

WHAT THE PLAYER EXPERIENCES

Where they can go and what they can see: the climb through the tower, the window that frames the moored rowboat, the island gate, a sea route that narrows toward the outpost, a lamp room that sees the ship, and sightlines to the beacon.

WHAT IT BUILDS

- Greybox lighthouse, sea corridor and outpost
- Interaction anchors for other agents
- Sightline checks for the window view and the ending

Review gate: traversal and sightline checks in the engine.

Gameplay / Systems agent

Serves pillars 1 and 2

WHAT THE PLAYER EXPERIENCES

The rules of the night: the story advancing in the right order, the deadline starting on the return crossing, fair time penalties for mistakes, and an ending that resolves once per attempt as a win or a fail.

WHAT IT BUILDS

- Phase controller and save state
- Repair jobs from data
- Sailors' arrival trigger, island gate, boarding and return-departure clock trigger
- Hidden deadline, cargo, win and fail

Review gate: the 10-minute slice plays end to end with no dead ends.

Hazard Forecast agent

Serves pillars 2 and 4

WHAT THE PLAYER EXPERIENCES

Jeff's warnings that always match the real threat, so the player can trust a call and read the route.

WHAT IT BUILDS

- Shared hazard forecast
- One safe-route signal
- Difficulty-based call timing

Review gate: Jeff's calls never disagree with the world; one route only until branching is approved.

Story / Fail-State agent Serves pillars 1, 3 and 4

WHAT THE PLAYER EXPERIENCES

The ship introduced at the boarding threshold, a fail they can see or hear, and a clear reason why. Ted waiting at the lamp.

WHAT IT BUILDS

- Fail sequence: horn, crash, silence, Failed screen
- Reason line
- Ship reveal and Ted's lamp staging

Review gate: a tester can say why they failed from every expiry location.

Lighting agent Serves pillars 1, 2 and 4

WHAT THE PLAYER EXPERIENCES

The beacon as both objective and guide home, warm lamplight against the storm, and darkness that deepens as time runs out while hazards stay readable.

WHAT IT BUILDS

- Beacon sweep and dynamic lights
- Storm visibility presets by phase
- Minimum hazard-visibility rules

Review gate: the beacon and hazards stay legible at every phase.

Sound agent Serves pillars 2 and 4

WHAT THE PLAYER EXPERIENCES

A storm they can hear getting worse, which tells them time is short without a timer, and Jeff's voice cutting through it when they need a warning. No shock stingers.

WHAT IT BUILDS

- Layered wind, rain, machinery, foghorn and ship impact
- State-based mix and escalation
- Source list and licences

Review gate: Jeff's warnings remain audible at peak storm.

Asset agent Serves pillars 3 and 4

WHAT THE PLAYER EXPERIENCES

The physical objects they handle during repairs, and the unsettling details (symbols, shrines, out-of-date posters) that make the lighthouse feel wrong.

WHAT IT BUILDS

- First-pass props and set dressing
- Source meshes and import notes
- Provenance records for every asset

Review gate: assets import cleanly and fit the approved reference sheet.

Dialogue agent Serves pillars 3 and 4

WHAT THE PLAYER EXPERIENCES

Ted's directions during repairs and his voice at the lamp, Jeff's calls on the water, and the journal's objectives. Together these are how the player knows what to do without markers, and they get sparser on harder settings.

WHAT IT BUILDS

- Lines for Ted, Jeff and Daniel as data
- Subtitle and difficulty variants
- Journal text

Review gate: the developer approves voice and lore before anything is recorded.

QA agent Serves all pillars

WHAT THE PLAYER EXPERIENCES

A night with no dead ends: every phase finishes, both outcomes can be reached, and mistakes are recoverable.

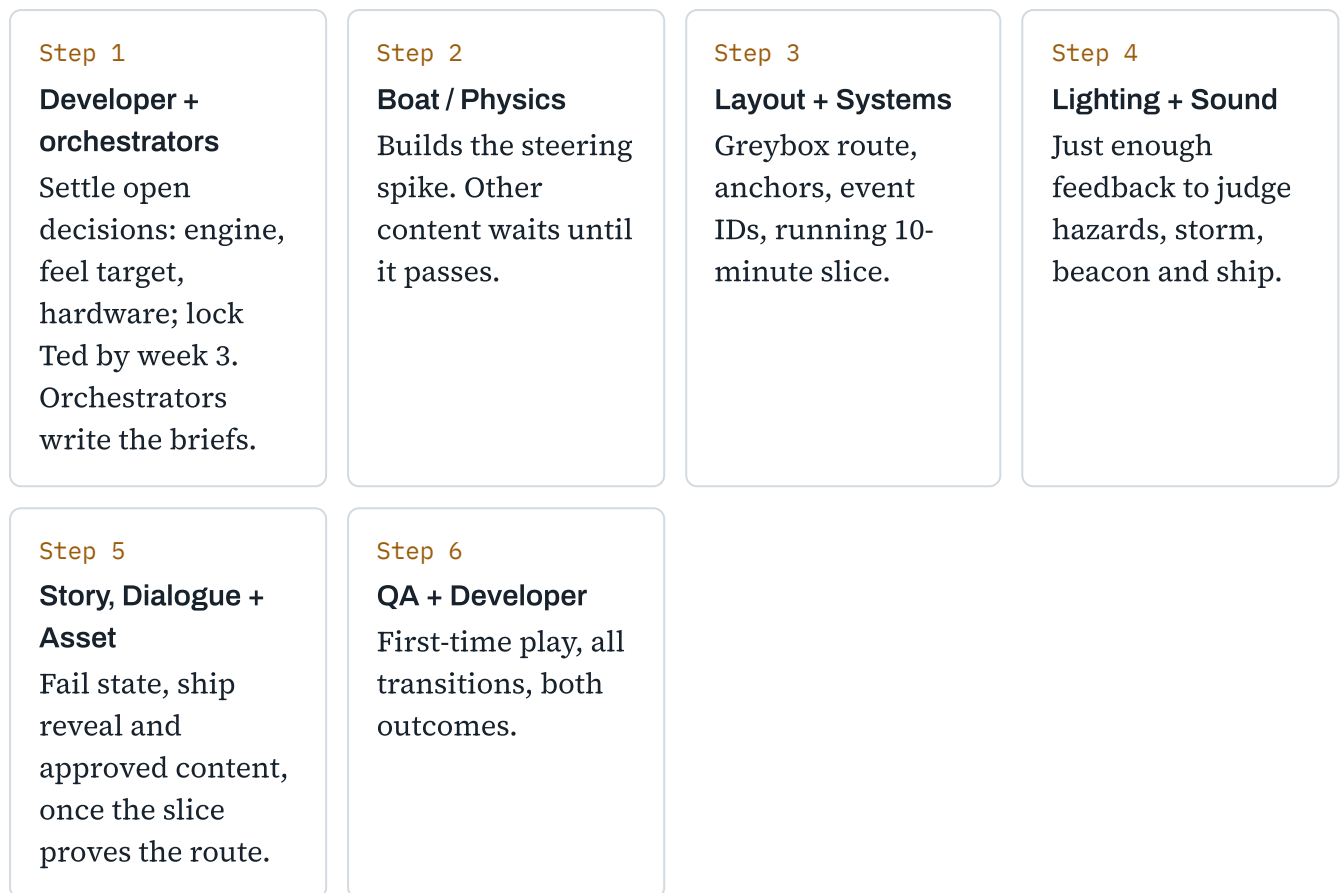
WHAT IT BUILDS

- Reproducible scenario list
- Bug reports against acceptance checks
- Checks with waves on and off

Review gate: the developer reproduces and prioritises each finding.

These are roles, not ten agents running at once. One agent can take successive bounded tasks.

Build sequence



Handoff review

Every handoff returns editable source in Godot text files and JSON data, with a HANDOFF.md note covering licences for any third-party material, test instructions and known limitations, and is checked against the same questions:

- Does it run in the integrated project from a clean checkout?
- Are state changes and event IDs consistent with the shared contract?
- Does it pass its task's acceptance checks?
- Does it pass each pillar's test and the scope rule, without deciding an unresolved story point, and respect the decisions log?
- What did human playtesting show about feel, clarity and pacing?
- Is there a complete HANDOFF.md, with data in the agreed file types, and has the orchestrator reported the result including failures and blocked decisions?

Handoff format

Godot stores scenes, scripts and resources as plain text, so agents can read, edit and diff them and the developer can review each change set.

What is handed off	Format	Notes
Scenes, scripts, resources	Godot text files: .tscn, .gd, .tres	One change set per task; no edits to another agent's files
Game data	JSON or .tres with stable IDs	Repairs, dialogue, journal, difficulty presets, hazard placement
Source art and audio	Blender .blend and glTF for meshes; WAV for audio; .import settings	Provenance and licence recorded per asset
Task handoff note	HANDOFF.md, one per task	Written by the worker agent
Orchestrator report	REPORT.md, per task or per gate	Written by the orchestrator from the HANDOFF.md notes
Contract changes	Proposal in the HANDOFF.md, then a documentation update once accepted	Event names, save IDs, state contract, hazard-forecast interface

HANDOFF.md headings, in order: task and goal; files changed; how to run and test; observed behaviour; not verified; licences and provenance; interface or contract changes proposed; open creator decisions. **REPORT.md headings:** what was delegated; what came back; what passed review; what failed and why; what remains; blocking creator decisions.

- A handoff without a HANDOFF.md is incomplete and goes back to the worker.
- Use the stable IDs in the shared contract; do not invent new IDs without raising a contract change.
- Keep notes short and factual: what was observed, not what was intended.

Technical Strategy

Engine, build contract and schedule; how the agent roles are organised; the token budget; and the API and plan constraints the work runs under.

Engine

Confirmed: Godot 4 with GDScript. It is free with no royalties, iterates quickly on modest hardware, and stores scenes as plain text, so AI agents can read and edit scenes as well as scripts. Its main weakness, no built-in ocean, is off the critical path now that waves are a stretch goal.

Factor	Godot 4	Unity	Unreal 5	Phaser
First-person 3D	Good	Very good	Excellent	Not supported (2D)
Steering past hazards	Straightforward	Straightforward	Straightforward	2D only
Waves (stretch)	Build yourself	Asset Store packages	Built-in, realism-focused	2D only
Storm lighting	Adequate with effort	Very good	Excellent	Poor
AI can edit scenes	Yes (text)	Partly (YAML)	No (binary Blueprints)	Yes (all code)
Iteration speed	Fast	Medium	Slow	Fastest
Licence	Free, MIT	Free below revenue threshold	Royalty above high threshold	Free, MIT

Unity was the alternative. Unreal is ruled out because Blueprints shut out AI help and iteration is slow. Phaser cannot meet the first-person brief (pillar 3).

Build contract

- **Separately owned scenes:** lighthouse, sea, outpost, shared player and interaction scenes, and one central game-state controller.
- **Content as data** with stable IDs: repairs, dialogue, assist presets and journal text.
- **Explicit phases:** LIGHTHOUSE_REPAIRS → SHELTER_AND_GATE → BOARDING → OUTBOUND → OUTPOST → RETURN → FINAL_REPAIR → RESOLUTION (WIN | FAIL)
- **Invariants:** the deadline is inactive until the return-departure event; Jeff is the only companion; pickup precedes return; the outcome resolves exactly once per attempt. “Unlocked routes” is removed from the save contract.

- **Two weather channels:** perception (sound, lightning, wind, the ship's lights) follows elapsed time; hazard density and route width follow phase and position.
- **Retry:** Gate 1 restarts the slice on a fail. Gate 2 settles the final model, in which a snapshot restores time and weather together.
- **Boat handling:** docking at the jetties is automatic; leaving the corridor triggers a recovery to the last safe point at a time cost.
- **Events, not side effects:** audio, UI and dialogue respond to game events and never change progression directly.
- **Shared hazard forecast:** the hazards, the boat's reaction and Jeff's calls read from one source, so warnings always match the real threat.

Milestones

Gate	Deliverable	Acceptance
0 · Steering spike	Greybox boat, hazards, heading and collision feedback, hazard forecast	Steering alone completes a short route; tunable without rewrites. Stop or redesign if not enjoyable.
1 · 10-minute slice	Two repairs, shelter and gate, both crossings, and win and fail outcomes with a Failed screen	Playable end to end; deadline starts when the return crossing begins; mistakes cost time but never trap the run; a fail restarts the slice and says why.
2 · Narrative alpha	All repairs, lighthouse, island and outpost discoveries, full dialogue, final retry and save model	Every story state can be finished; no dead-end saves or lost cargo; a reload restores time and weather together.
3 · Content and polish	Final art, audio, accessibility, checkpoints	New players show legible navigation and 30-minute pacing.

Six-week prototype schedule

Week	Focus	Exit test
1	Project setup and pipeline; boat at fixed speed with one steering axis; greybox sea; first rocks, buoys and debris	The boat steers and collides reliably
2	Steering feel, hazard readability, collision feedback and recovery, shared hazard forecast (Gate 0)	Go/no-go: steering past hazards is fun on its own
3	Outpost and burner pickup; harder return crossing; storm visibility and lighting; docking	Both crossings playable end to end; Ted's presence locked

Week	Focus	Exit test
4	First repair, Ted at the window, island gate, Jeff at the boat, Jeff's calls, return-crossing clock trigger, first deadline tuning pass	The full 10-minute path is playable and tested by an outsider
5	Second repair, hidden deadline, fail presentation, Ted's lamp lines, win and fail outcomes, placeholder audio	Both outcomes reachable; the fail reason is shown
6	Playtests with 3–5 new players, tuning, bug fixing, tester build (Gate 1)	Prototype outcome checks pass

Week 5 is heavily loaded. If it slips, the developer decides between dropping the second repair for Gate 1, cutting waves (already first to go), or extending the schedule.

Waves are not in this schedule. They start only after Gate 0 passes and the current week's exit test is met (earliest: spare time in week 5), sit behind one on/off setting, and are cut first if anything slips.

Agent roles and working rules

The ten worker roles in Part 3, supervised by the two orchestrators, map onto the build contract as follows. Boat/Physics and Gameplay/Systems agree event names before working in parallel; Level Layout supplies scene anchors before Lighting, Sound and Asset integrate.

Agent	Owns	Input contract
Boat / Physics	Boat scene, controller, tuning resource	Hazard forecast API, one steering axis, recovery event
Level Layout	Greybox scenes, anchors	Location sizes, beacon visibility, interaction anchors
Gameplay / Systems	Phase controller, save state, repairs, deadline	Stable event and save IDs
Hazard Forecast	Shared forecast, safe-route signal	Route data, hazard placement, difficulty presets
Story / Fail-State	Fail presentation, ship reveal, Ted's lamp staging	Fail-state contract, phase events, decisions log
Lighting	Light rigs, visibility presets	Scene anchors, performance budget, visibility rules
Sound	Audio banks, mix states	Escalation events, speaker priority
Asset	Source meshes, import settings	Scale, pivot, collision, palette, reference sheet

Agent	Owns	Input contract
Dialogue	Dialogue and journal data	Character voices, phase events, ambiguity rule
QA	Scenario list, bug reports	Playable build, event log, acceptance checklist

- Orchestrators write the briefs, order the dependencies and report results; only the developer merges.
- One branch or change set per task; no agent overwrites another’s scene, script or asset.
- Contract changes are raised before any dependent file is edited.
- Each task runs the project build and focused checks, and reports what was observed, what could not be verified, and which human decision remains.

Token budget

Estimated from a unit of one **agent task**: about 40 model turns at an average 60K-token context, giving about 2.4M input tokens (about 90% cache reads) and 40K output tokens. Full-game tasks after the prototype assume a larger 90K context as the codebase grows.

~\$500
Prototype · ~250 tasks ·
range \$250–\$1,400

~\$2,050
Full game incl.
prototype · ~835 tasks ·
range \$1,000–\$5,000

~20M
Input tokens per
working day during the
prototype, mostly
cached

Model	Role in the mix	Input / MTok	Output / MTok	Cache read	Per task
Claude Opus 5.5	Implementation and debugging (70%)	\$4.00	\$20.00	\$0.20	\$2.36
Claude Sonnet 5	Routine scripting, data, docs (20%)	\$2.00	\$10.00	\$0.20	\$1.40
Claude Haiku 4.5	Search, file sweeps, formatting (10%)	\$1.00	\$5.00	\$0.10	\$0.70
Claude Fable 5.1	Not planned; comparison only	\$10.00	\$50.00	\$0.25	\$5.36

Prototype workstream	Tasks
Planning and docs	15
Project setup and asset pipeline	10
Boat, hazards, steering and tuning	40
Stretch: wave hazards, only if time allows	15
Greybox levels and sightlines	20
Interaction, repairs, shelter and gate	30
Jeff's calls and placeholder dialogue	20
State machine, deadline, endings and save	20
Lighting, weather and audio integration	30
Bug fixing and integration	40
Playtest analysis and tuning	10
Total	250

Not included: AI image, 3D, music or voice tools billed by other vendors; paid engine assets; the Steam Direct app fee. Real usage will be measured after week 1 and the forecast revised. These figures have not been re-baselined for the post-review changes (second repair, fail-state work and orchestrator overhead).

API and plan constraints

Constraint	Effect on the project
Development-time only	No runtime AI in the shipped game, so no API keys, latency, per-player cost or availability risk after release.
Plan usage limits	Development runs on a Claude Team plan with rolling 5-hour and weekly limits. Extra usage is off, so hitting a limit pauses work until it resets. The prototype pace is estimated at about 30–50% of the weekly limit; this is checked at the end of week 1.
Context windows	Opus 5.5 and Sonnet 5 accept up to 1M tokens; Haiku 4.5 accepts 200K. Narrow task briefs keep contexts far below this, which also keeps cost down.

Constraint	Effect on the project
Prompt caching	Caching matches on the exact prefix; any change to earlier content invalidates everything after it. Stable project docs and briefs make most input cheap cache reads (writes cost about 1.25× input).
Opus 5.5 behaviour	Thinking cannot be switched off; depth and spend are controlled with the effort setting, which defaults to medium. Routine tasks run at lower effort.
Cost levers	Route routine work to Sonnet 5 or Haiku 4.5, keep contexts small, and start fresh sessions between unrelated tasks.
Asset rights	Every AI-generated or third-party asset carries a provenance and licence record before it is merged.

Feasibility

Target	6 weeks, solo, AI-assisted	Reasoning
10-minute prototype	Feasible, with conditions	A greybox build for playtesters, not a product to sell. It depends on steering passing in week 2, frozen scope, placeholder art and audio, and week 6 kept for playtests.
30-minute full game	Not feasible	Estimated 27–31 weeks solo (about 5 months with stock assets and AI voice). The largest gaps are 3D art and character animation, QA, and Steam launch preparation, including a public Coming Soon page for at least two weeks before release.

AI realistically speeds up code, data, docs and dialogue drafts by 1.3–2×. It helps little with steering feel, visual readability, consistent 3D art and playtesting. The plan therefore commits the six weeks to the prototype, and uses the week-2 steering result and week-6 playtests to decide whether to fund the full game.

Risks

1. **Boat feel** is the largest risk. Making waves a stretch goal removes its hardest part from the critical path, and Gate 0 tests it first.
2. **Scope creep** across repairs, sequences and environmental detail, held back by the pillars and the scope rule.
3. **Usage limits** interrupting work, monitored weekly and handled by routing routine tasks to smaller models.

4. **Week 5 load:** the second repair, deadline, fail state and Ted's lamp lines land together; the developer decides what gives if it slips.

Open decisions

Settled in the design review, then what is still open. Proposed answers are in use until confirmed.

Decided in the review

Area	Decision	Outcome
Deadline	Retry at Gate 1	Fail, then restart the slice; final model before Gate 2
Deadline	Storm escalation	Perception follows elapsed time; hazards follow phase and position
Deadline	Clock start	When the burner is aboard and the return crossing begins; no fail before then; ship introduced at the boarding threshold
Deadline	Fail presentation	Horn, crash into the island, silence, Failed screen with a reason; three fail states, all after the return crossing begins
Story	Sailors' arrival	No crash; they moor on the island seeking shelter; the player opens the island gate
Story	Daniel after the gate	In the basement; no reaction recorded
Story	Outpost detail	None significant until the prototype is complete; extra clues parked in Dreams for the game
Story	Discoveries	Core set in the lighthouse and island, before boarding; lighter set at the outpost
Boat	Return-crossing shortcut	Dropped for now; Gate 2 cut-list candidate
Boat	Crossing penalty rules	Loose for Gate 1; reviewed for the full game
Production	Repairs at Gate 1	Two
Production	Engine	Godot 4 with GDScript, confirmed

Still open

Area	Decision	Current proposal
Story	Ted's presence	Lock by end of week 3; present, staged at the lamp, voice and position only
Boat	Outbound crossing penalties	No deadline cost before the return crossing; deferred, to be decided later
Deadline	Length	Set from playtests, with room for some mistakes; reviewed once the return crossing from the outpost is tested
Deadline	Final retry and save model	Before Gate 2; a snapshot that restores time and weather together
Deadline	Pause rules	Stops in menus and pause; check for exploits before Gate 2
Boat	Docking and recovery	Proposed: automatic docking; recovery to the last safe point
Boat	Bailing interaction	Only if playtests show it helps
Production	Target hardware and feel target	To define before Gate 0
Production	Controller support in the prototype	Open
Production	Which two repairs are in the prototype	Open
Production	Full-game estimate and cut list	Complete after Gate 1
Stretch	Wave hazards	Decide at the end of week 4

Beyond scope

Ideas deliberately kept out of the prototype and full game. None is scheduled; each would need to pass the pillars first.

- **Return-crossing shortcut.** A riskier branching route on the way back, giving the pressure an outlet and repairs a mechanical output. Deferred from Gate 1; first candidate on the Gate 2 cut list.
- **Companion choice.** Choose Jeff or Daniel before boarding; either can complete both crossings.
- **Daniel as a companion.** A flashlight that is faster to read than Jeff's voice but occasionally wrong. It conflicts with pillar 4, so it would need playtesting first.
- **Hidden coda.** An ending unlocked by journal fragments, shrines and symbols, revealing what happened to earlier keepers while leaving the supernatural unconfirmed.
- **Full Gale mode.** A New Game+ run that remixes the repairs and crossings into a harder, repeatable shift.